Daemon Thread [Service Level Thread]

Daemon thread is used to provide background maintenance.

The main purpose of of Daemon thread to provide services to the user thread.

JVM can't terminate the program till any of the non-daemon (user) thread is active, once all the user thread will be completed then JVM will automatically terminate all Daemon threads, which are running in the background to support user threads.

The example of Daemon thread is Garbage Collection thread, which is running in the background for memory management.

In order to make a thread as a Daemon thread as well as to verify whether a thread is daemon thread or not, Java software people has provided the following two methods

1) public final void setDaemon(boolean on) : If the boolean value
   is true the thread will work as a Daemon thread.

2) public final boolean isDaemon() : Will verify whether the
   thread is daemon thread OR not (User thread)

//Program

```java
 package com.ravi.daemon;

 public class DaemonThreadDemo1
 {
   public static void main(String[] args)
     {

         IO.println("Main Thread Started...");

         Thread daemonThread = new Thread(()->
         {
             while(true)
             {
                 IO.println("Daemon thread is running");
                 try
                 {
                     Thread.sleep(2000);
                 }
                 catch(InterruptedException e)
                 {
                 }
             }
         });
         daemonThread.setDaemon(true);
         daemonThread.start();

         Thread userThread = new Thread(() ->
         {
             for(int i=1; i<=20; i++)
             {
                IO.println("User thread is running");
                try
                {
                    Thread.sleep(1500);
                }
                catch(InterruptedException e)
                {
                    e.printStackTrace();
                }
             }
             IO.println("USER THREAD COMPLETED!!!!");
         });

         userThread.start();

         IO.println("Main Thread Ended...");

     }
 }
```

Note : In the above program, first user thread will be completed then only JVM will automatically terminate the daemon thread.

```java
package com.ravi.daemon;

public class DaemonThread2
{
    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        IO.println(t.isDaemon()); //false

        t.setDaemon(true); //java.lang.IllegalThreadStateException
        System.out.println(t.isDaemon());

    }

}
```

We cannot make main thread as a daemon thread.

---

public void interrupt() method of Thread class :

It is a predefined non static method of Thread class. The main purpose of this method to disturb the execution of the Thread, if the thread is in waiting or sleeping state.

Whenever a thread is interupted then it throws InterruptedException so the thread (if it is in sleeping or waiting mode) will get a chance to come out from a particular logic.

Points :-

If we call interrupt method and if the thread is not in sleeping or waiting state then it will behave normally.

If we call interrupt method and if the thread is in sleeping or waiting state then we can stop the thread gracefully.

*Overall interrupt method is mainly used to interrupt the
thread safely so we can manage the resources easily.

Methods :
----

1) public void interrupt () :- Used to interrupt the Thread but the thread must be in sleeping or waiting mode.

2) public boolean isInterrupted() :- Used to verify whether thread is interrupted or not.

//Programs

```java
package com.ravi.interrupt;

class Test implements Runnable
{
    @Override
    public void run()
    {
        boolean isInterrupted = Thread.currentThread().isInterrupted();

        IO.println("Is Thread Interrupted :"+isInterrupted);

        for(int i=1; i<=10; i++)
        {
            IO.println("i value is :"+i);
        }
    }
}

public class InterruptDemo1
{

    public static void main(String[] args)
    {
        Thread t1 = new Thread(new Test());
        t1.start();

        //main thread is interrupting the child thread
        t1.interrupt(); //isInterrupted = true;

    }

}
```

Note : In the above program main thread is interrupting child thread so interrupt flag value will become true, now once the child thread(in this program) will go into sleeping OR waiting state then child thread will throw InterruptedException and interrupt flag value will become false but in this program the child thread is not in sleeping OR waiting state hence it will not throw any exception

---

```java
 package com.ravi.interrupt;

 class Interrupt extends Thread
 {
    @Override
    public void run()
     {
        boolean isInterrupted = Thread.currentThread().isInterrupted();
        IO.println("Is thread Interrupted :"+isInterrupted);

        for(int i=1; i<=5; i++)
         {
            System.out.println(i);

            try
            {
             Thread.sleep(5000);  //InterruptedException + isInterrupted => false

            }
            catch (InterruptedException e)
            {
                IO.println("Thread has Interrupted");
                Thread.currentThread().interrupt(); //isInterrupted = true
            }

         }
    }
 }
 public class  InterruptDemo2
 {
     public static void main(String[] args)
     {
         Interrupt it = new Interrupt();
         it.start();
         it.interrupt(); //main thread is interrupting the child thread

     }
 }
```

```java
 package com.ravi.interrupt;


 class Foo implements Runnable
 {
     @Override
     public  synchronized void run()
     {
         try
         {
         while(!Thread.currentThread().isInterrupted())
         {
             IO.println("Child thread has acquired the lock");
             Thread.sleep(1000);
         }
         }
         catch(InterruptedException e)
         {
             IO.println(e);
         }
         finally
         {
             IO.println("Resourse is released here");
         }
     }
 }

 public class InterruptDemo3
 {

     public static void main(String[] args)
     {
         Foo f1 = new Foo();

         Thread thread = new Thread(f1);
         thread.start();

         try
         {
             Thread.sleep(5000);
         }
         catch(InterruptedException e)
         {

         }
         //If main thread will not interrupt child thread then lock will not be available for main 
 thread
         thread.interrupt();

         synchronized(f1)
         {
             IO.println("Main method syn block");
         }

     }

 }
```

Deadlock :

```
                 t1            t2

                 +             +

                 R1            R2

                 +             +

                 R2            R1
               ___________________
         END
```

```
Thread t1 = new Thread();

synchronized(r1)
{
  Thread.sleep(1000);

   //Nested block
   synchronized(r2)
   {

   }
}
```

```
Thread t2 = new Thread();

synchronized(r2)
{
  Thread.sleep(1000);

    synchronized(r1)
    {

    }

}
```

It is a situation where two or more than two threads are in blocked state forever, here threads are waiting to acquire another thread resource without releasing it's own resource.

This situation happens when multiple threads demands same resource without releasing its own attached resource so as a result we get Deadlock situation and our execution of the program will go to an infinite state as shown in the diagram.

```java
 package com.ravi.interrupt;

 public class DeadlockExample
 {
   public static void main(String[] args)
   {
      String resource1 = new String("Java");  //(R1)
      String resource2 = new String("HTML"); //(R2)

    // t1 tries to lock resource1(R1) then resource2(R2)

    Thread t1 = new Thread()
     {
      @Override
      public void run()
          {
              synchronized (resource1)
              {
               IO.println("Thread 1: locked resource 1");
                  try
                  {
                  Thread.sleep(1000);
                  }
                  catch (Exception e)
                  {}


                   //t1 thread is waiting here for Lock2 (BLOCKED STATE)
                  synchronized (resource2) //Nested synchronized block
                  {
                   System.out.println("Thread 1: locked resource 2");
                  }
             }
      }
    };


    // t2 tries to lock resource2 then resource1
    Thread t2 = new Thread()
    {
      @Override
      public void run()
      {
        synchronized (resource2)
            {
          System.out.println("Thread 2: locked resource 2");
              try
              {
              Thread.sleep(1000);
              }
              catch (Exception e)
              {}

              //t2 thread will wait for L1  (Resourse1) [BLOCKED STATE]
              synchronized (resource1) //Nested synchronized block
              {
                System.out.println("Thread 2: locked resource 1");
              }
         }
      }
    };
    t1.start();
    t2.start();
   }
 }
```